

Dynamic Web Scrapping & Data Extraction Report

Automated Multi-Layer Scrapping & Excel Report Generation for Crickslab Match Central

PROJECT TYPE	TARGET PLATFORM	OUTPUT DELIVERED
Python Automation / Web Scrapping	Crickslab (Match Central)	Formatted Excel (.xlsx) & Python Script

1. Executive Summary

This technical report documents the complete implementation of a robust, dynamic Python web scraper designed to extract detailed player profiles and career statistics from **Crickslab Match Central**. Standard scraping techniques often fail on modern sports platforms due to dynamic DOM rendering, client-side JavaScript execution, and nested detail structures.

The developed automation solution successfully circumvents these technical challenges by deploying a **Two-Step Scrapping Workflow** (Selenium Headless Execution + BeautifulSoup HTML Parsing) that navigates multi-level profile pages to systematically capture **32 individual career parameters** for player profiles.

32	4	2-Step	100%
Extracted Data Fields	Categorized Excel Tabs	Automated Scraper Pipeline	Data Cleaning Accuracy

2. Technical Architecture & Strategy

The web scraper was architected using a modular design to ensure high efficiency, reliability, and maintainability.

Component	Technology Used	Functional Role
Browser Automation	Selenium WebDriver (Chrome)	Executes client-side JS scripts, bypasses dynamic rendering, and scrolls DOM elements.
DOM Parsing	BeautifulSoup4 & Regex	Parses HTML source code and applies regular expressions to extract key metric fields.
Data Structuring	Pandas & OpenPyXL	Aggregates extracted dictionaries, formats multi-tab Excel workbooks, and applies data filters.
Driver Management	WebDriver Manager	Handles Chrome binaries automatically, eliminating manual driver configuration.

Core Challenge Solved: Dynamic JS & Multi-Level Navigation

Crickslab uses dynamic loading where player listings do not immediately expose complete stats. The scraper overcomes this by first harvesting unique player profile links from the main directory, then visiting each profile page sequentially to extract granular career numbers (Batting Avg, Bowling Economy, Strike Rates, 50s/100s, etc.).

3. Extracted Data Schema (32 Fields)

The solution extracts and maps data across four key statistical groups:

Profile & Demographics: Player Name, Site Role, Nationality, City, Batting Style, Bowling Style, Profile URL. **Career Totals:** Total Career Matches, Total Career Runs, Total Career Wickets. **Batting Metrics:** Batting Matches, Innings, Runs, High Score (HS), Batting Average, Strike Rate (SR), 50s, 100s, 4s, 6s. **Bowling & Fielding Metrics:** Bowling Matches, Innings, Overs, Maidens, Runs Conceded, Wickets, Economy, Bowling Avg, Bowling SR, 5W, Catches, Run Outs.

4. Deliverable Excel Structure

The output is compiled into a polished, professional workbook titled `Crickslab_Players_Data.xlsx` containing structured worksheets for effortless client review:

Worksheet Name	Description & Content Scope
All Players	Master sheet containing complete extracted records across all 32 parameters.
Batsmen	Filtered subset targeting players categorized under Specialist Batting roles.
Bowlers	Filtered subset highlighting players categorized under Specialist Bowling roles.
All Rounders	Filtered subset containing versatile players with significant dual stats.

5. Project Execution Instructions

To run the automation script on any fresh environment, follow these steps:

```
# Step 1: Clone or navigate to the project directory
cd /path/to/Crickslab_Projectfolder

# Step 2: Install necessary Python packages
python3 -m pip install --user pandas openpyxl bs4 selenium webdriver-manager

# Step 3: Execute the scraper script
python3 scraper.py
```